

Lists

Discovering Computer Science, Chapter 7.2

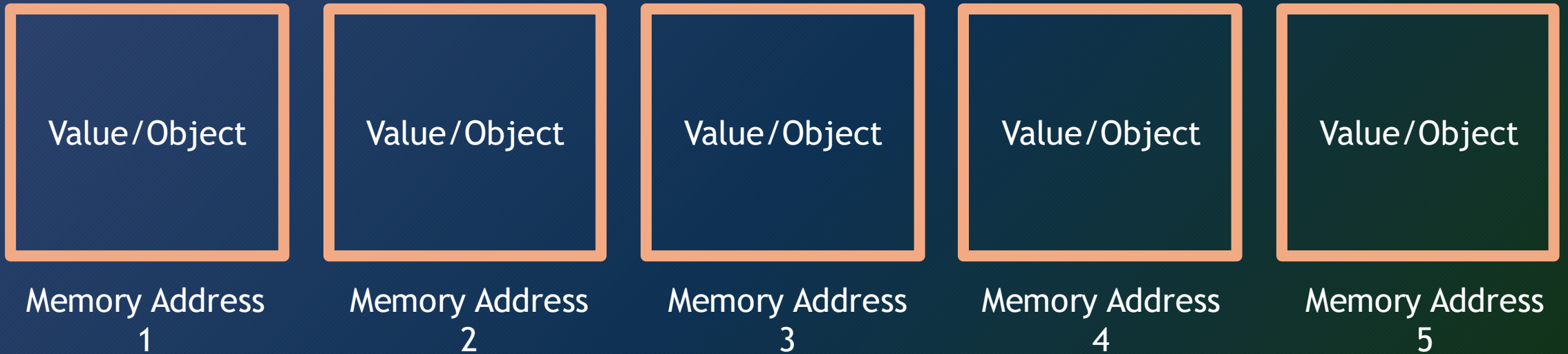
Part 0: Review

Setting a Convention: Computer Memory



A Memory Cell/Bucket

Setting a Convention: Computer Memory



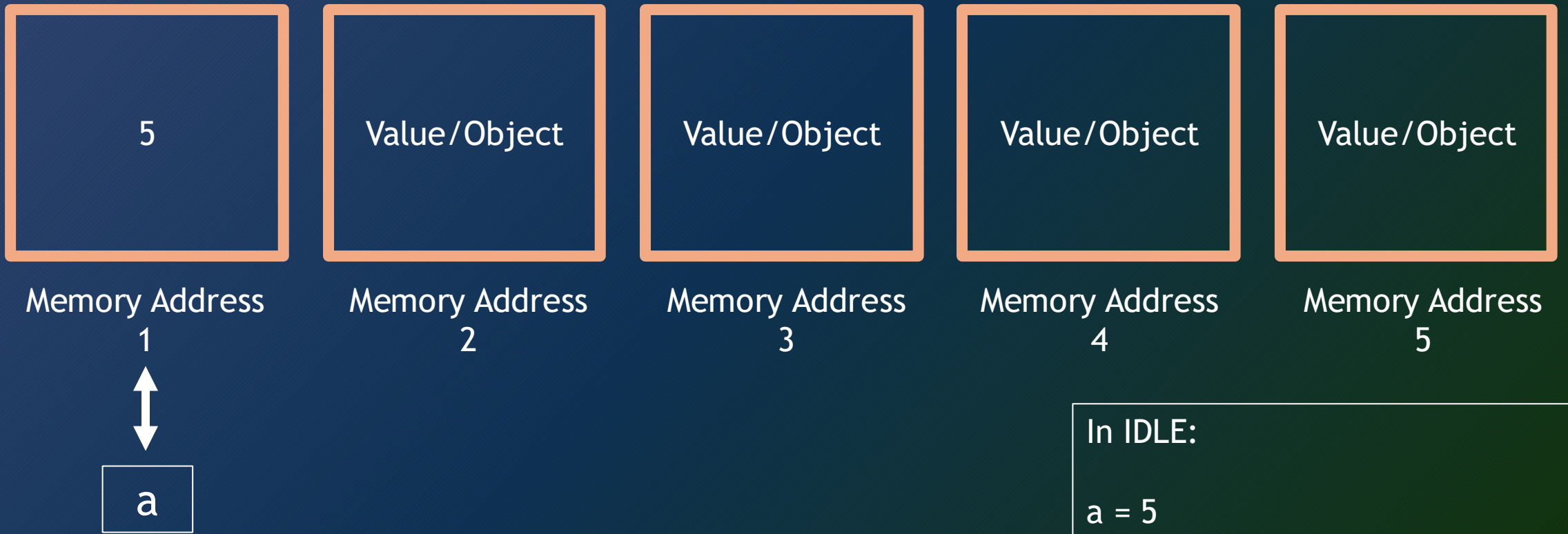
Review



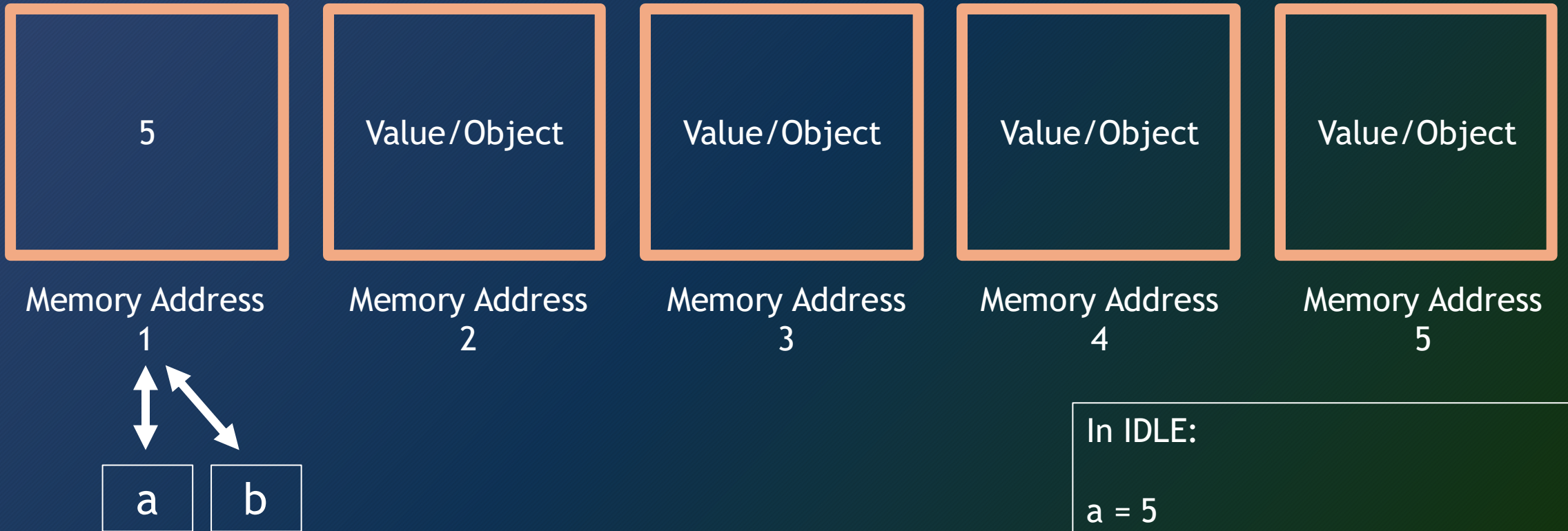
In IDLE:

a = 5

Review



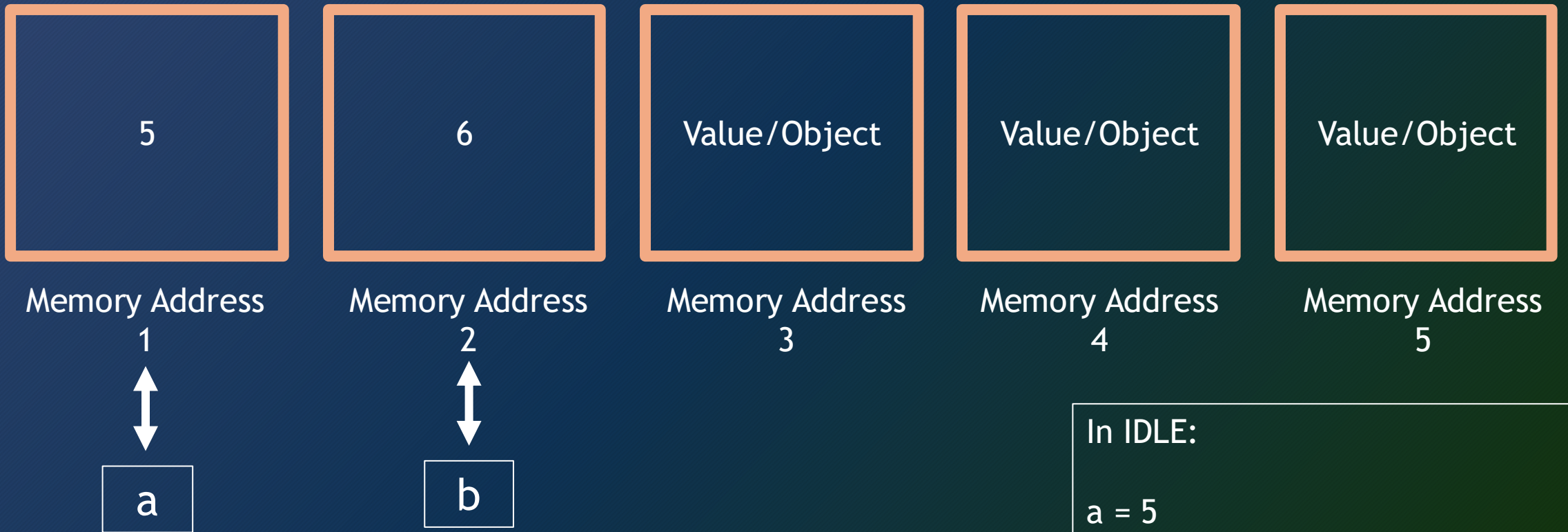
Review



In IDLE:

```
a = 5  
b = a
```

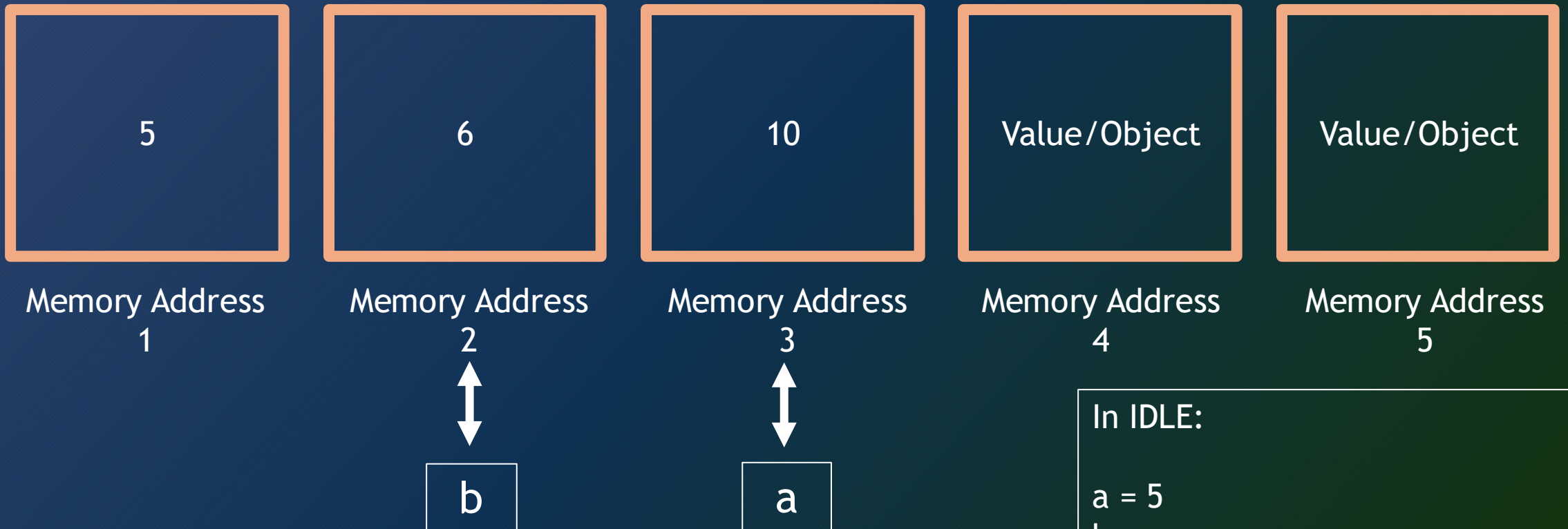

Review



In IDLE:

```
a = 5  
b = a  
b = b+1
```

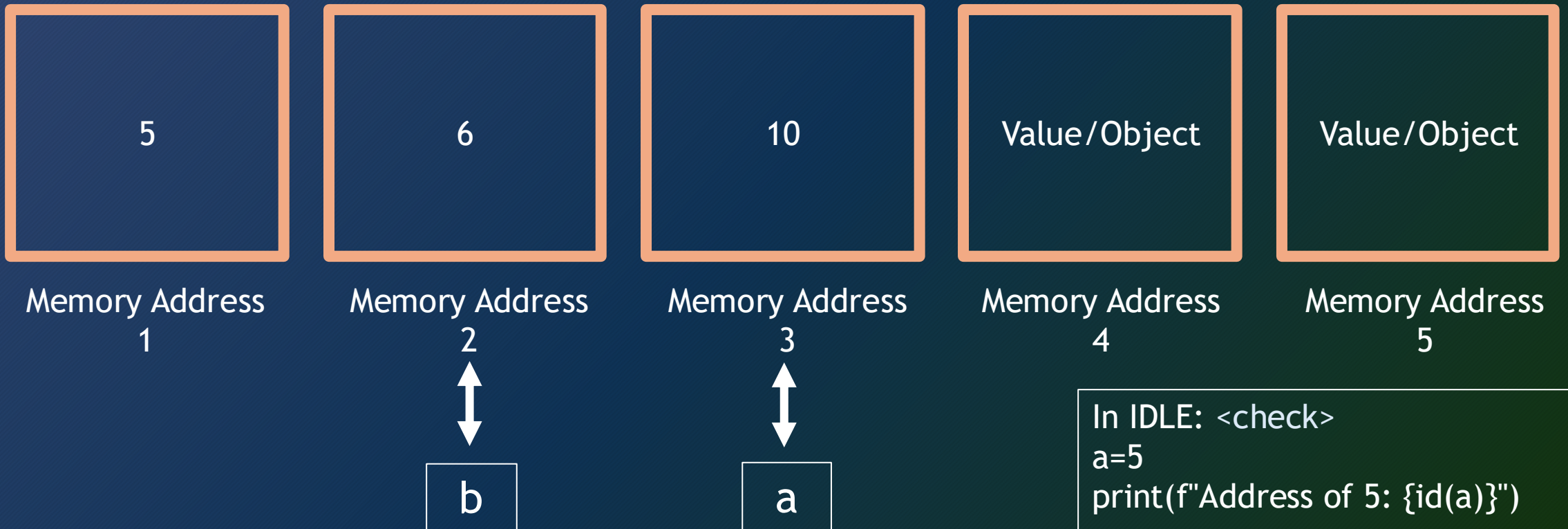

Review



In IDLE:

```
a = 5  
b = a  
b = b+1  
a = 10
```


Review



```
In IDLE: <check>
a=5
print(f"Address of 5: {id(a)}")

a=10
print(f"Address of 10: {id(a)}")
```


Review: Immutable Data Types

An **immutable data type** in Python is an object whose value or internal state cannot be modified after it is created.

Any operation that appears to change an immutable object actually results in the creation of a new object in memory, with the variable name then being reassigned to refer to this new object

Examples of Immutable Data Type:

- **Integers:** 5, 100, -1
- **Floats:** 3.14, 2.0
- **Strings:** "hello", "Python"
- **Tuples:** (1, 2, 3)
- **Booleans:** True, False

Part 1: Let's get to know Lists

The Mutable Container

Mutable

A **mutable data type** in Python is a type of object whose state or value can be modified after it has been created, without changing its unique identity (memory address)

This means we can add, remove, or change elements of a mutable object "in-place".

Lists - A collection of items

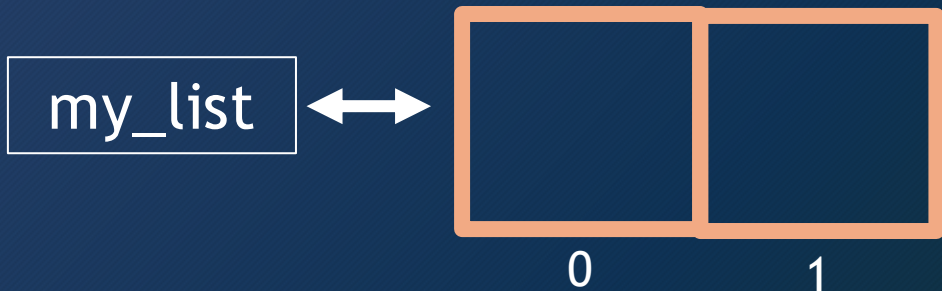


1. Lists Store References

In IDLE:

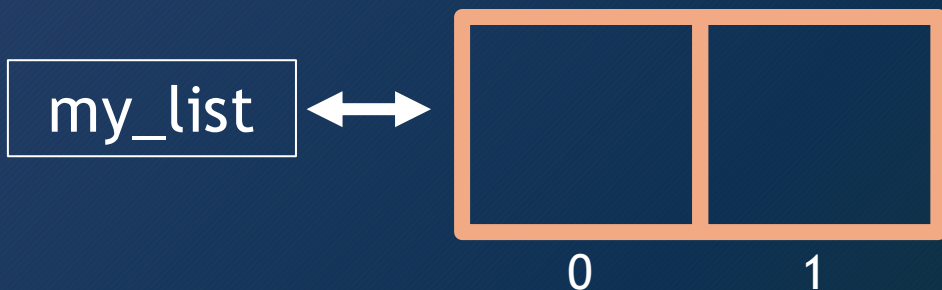
```
my_list = [100, 50]
```


1. Lists Store References



In IDLE:
`my_list = [100, 50]`

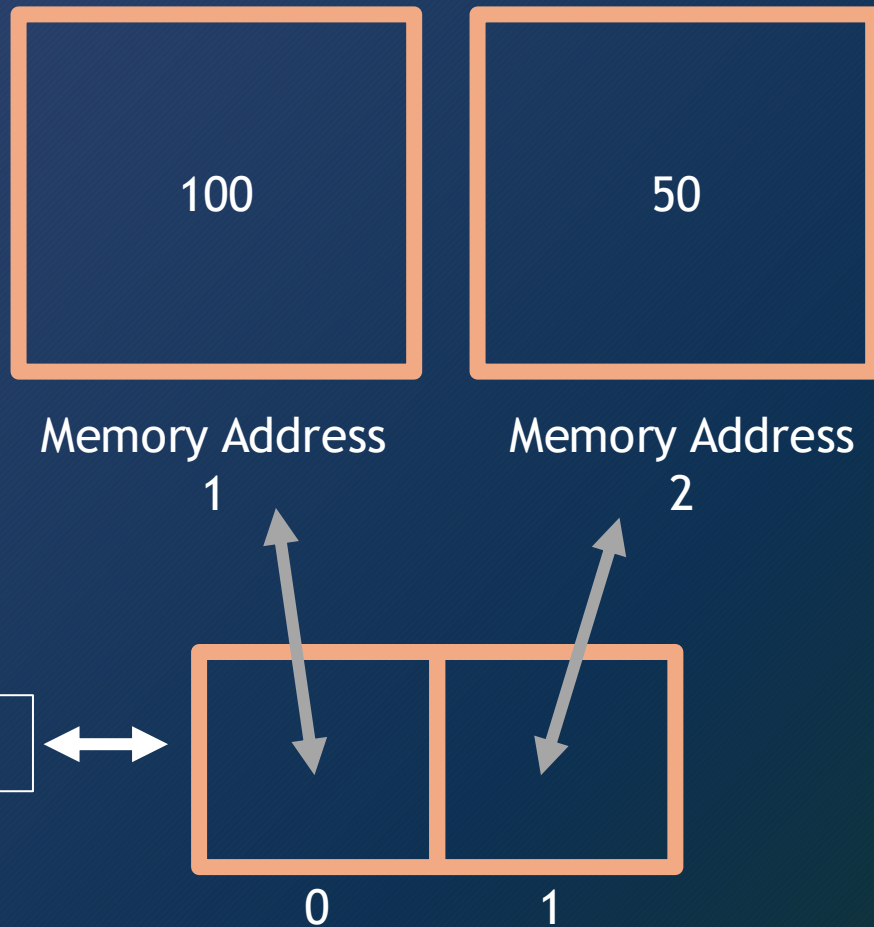
1. Lists Store References



In IDLE:

```
my_list = [100, 50]
```


1. Lists Store References



In IDLE:

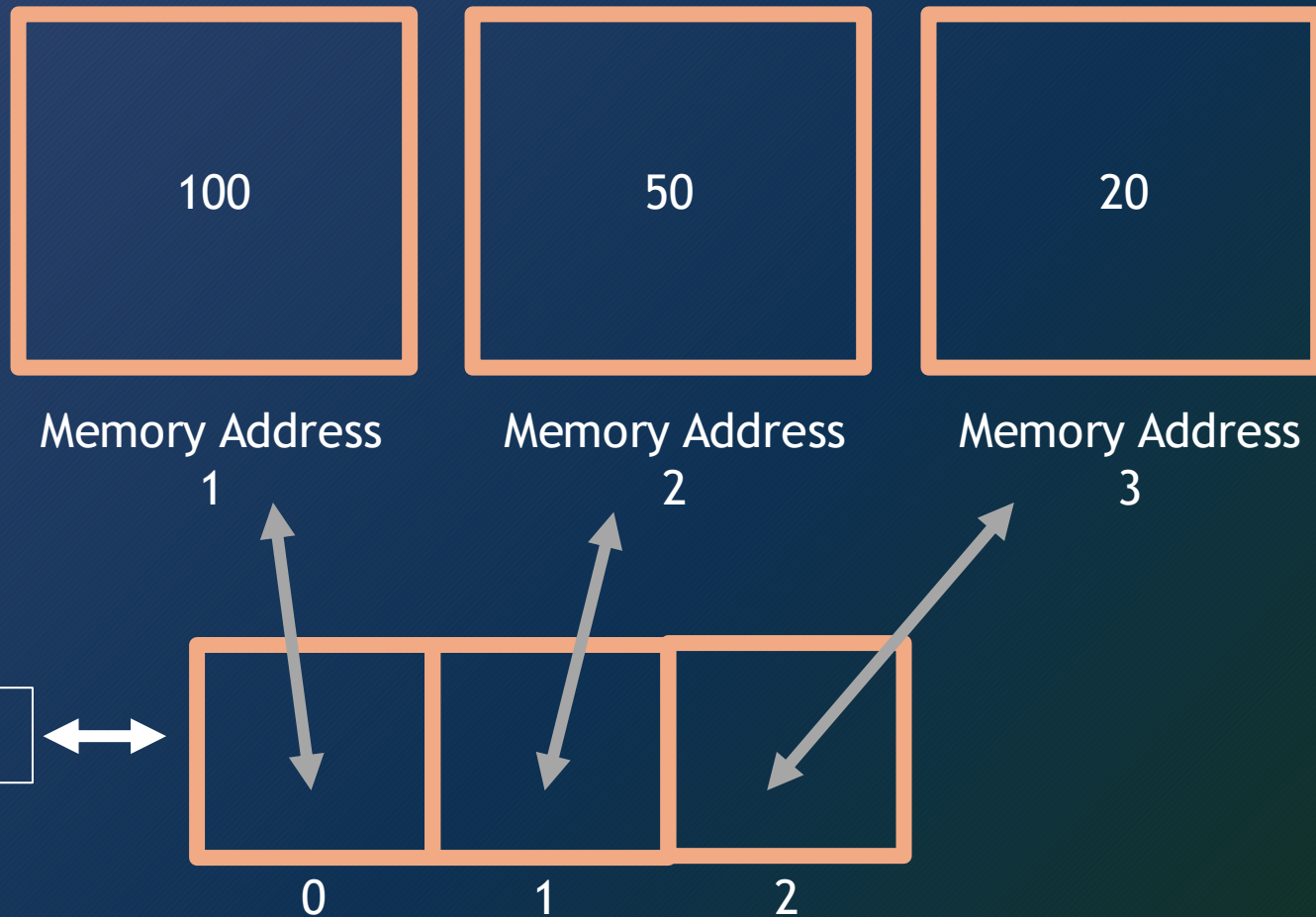
```
my_list = [100, 50]
```


2. Increasing The Length Of The List

In IDLE:

```
my_list = [100, 50]  
my_list.append(20)
```


2. Increasing the length of the list

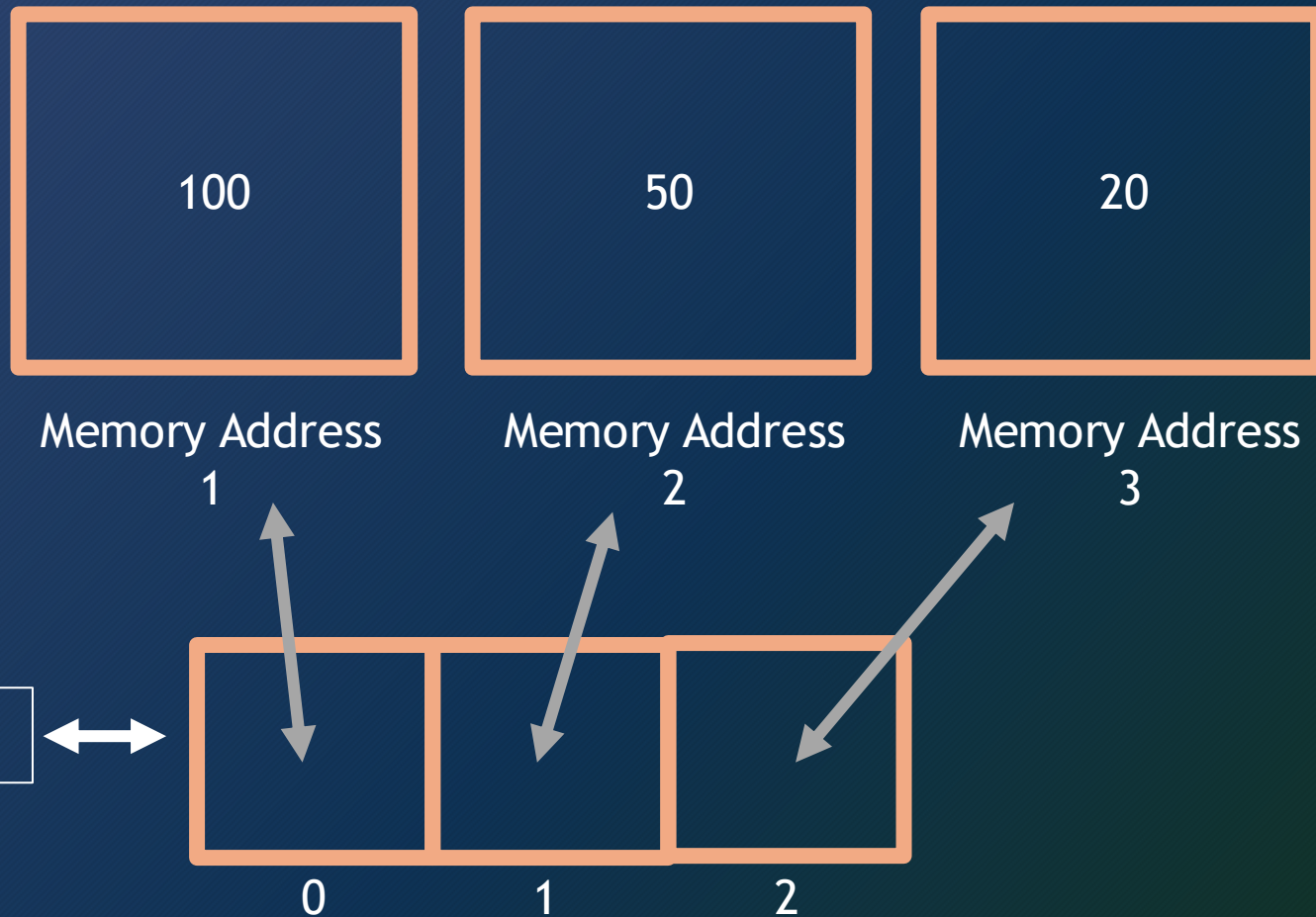


In IDLE:

```
my_list = [100, 50]  
my_list.append(20)
```


3. Accessing Data From A List

3. Accessing Data From A List



In IDLE:

```
print( my_list[0] ) # 100
print( my_list[1] ) # 50
print( my_list[2] ) # 20
```

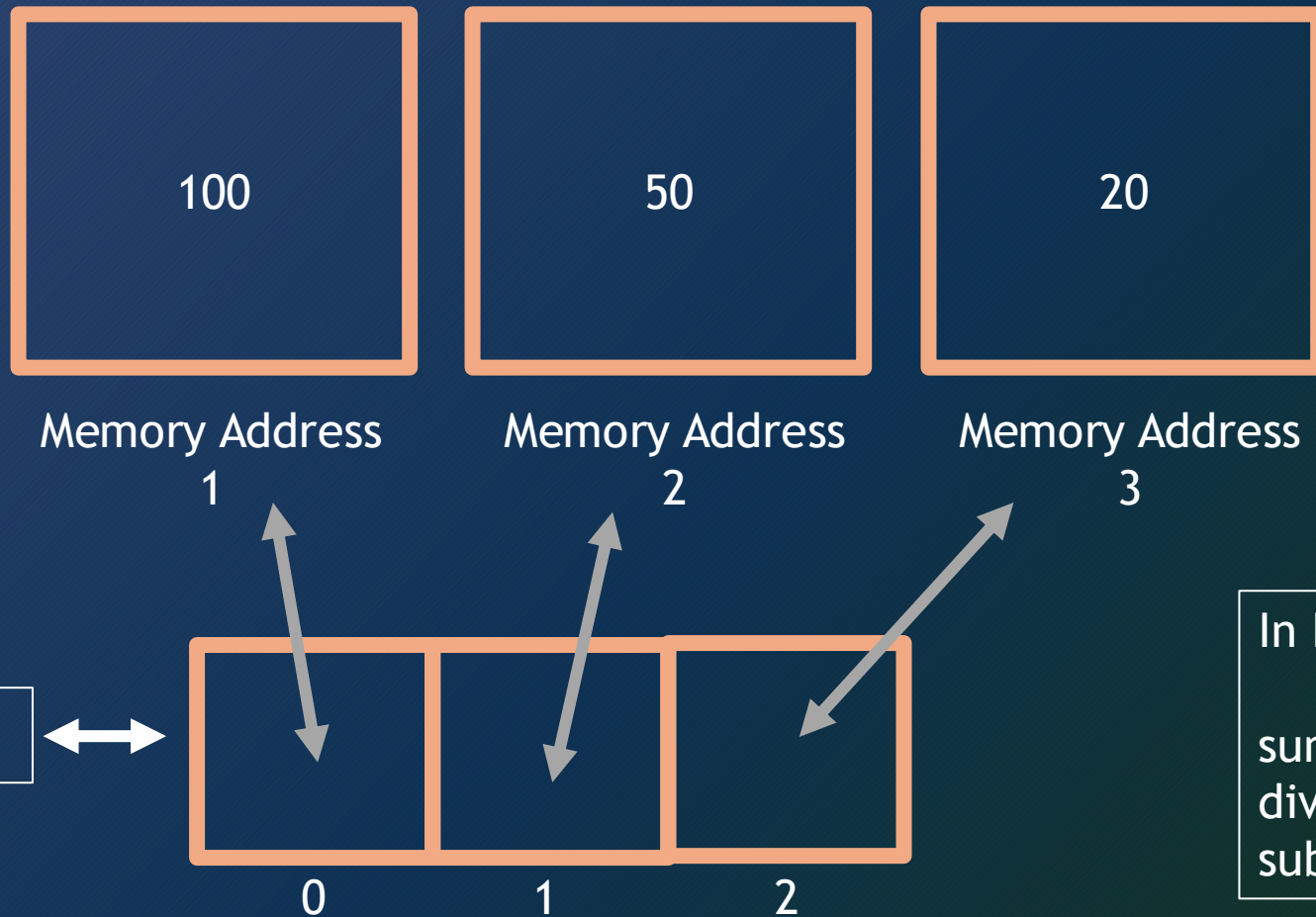

4. Calculating With Accessed Data

4. Calculating With Accessed Data

In IDLE:

```
sum = my_list[0] + my_list[0]  
subtract = my_list[0] - my_list[1]  
Division = my_list[0] / my_list[1]
```


4. Calculating With Accessed Data



In IDLE:

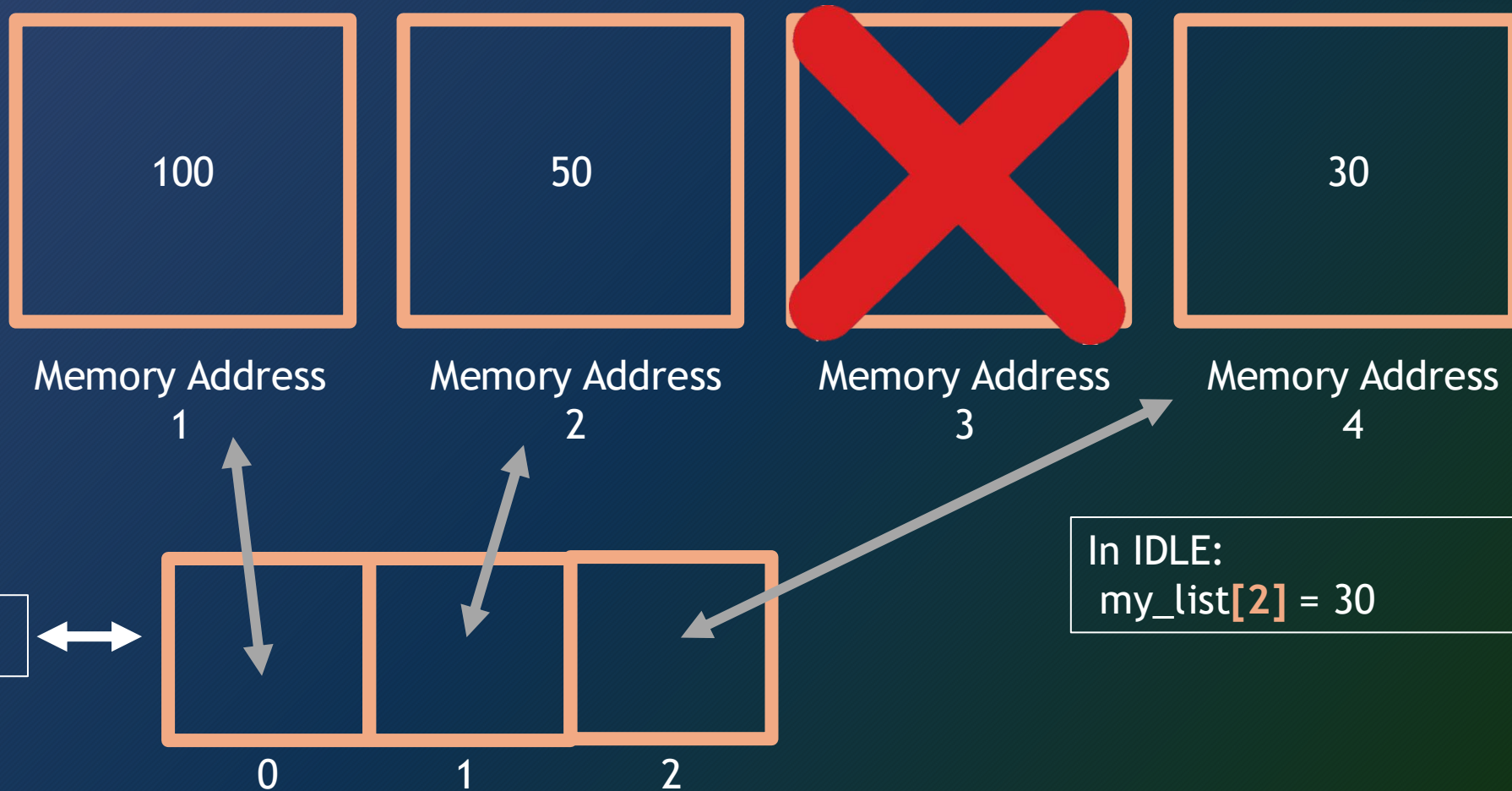
```
sum = my_list[0] + my_list[0] # 200
division = my_list[0] / my_list[2] # 5
subtract = my_list[1] - my_list[2] # ???
```


5. Modifying An Existing Value/Item

5. Modifying An Existing Value/Item

```
In IDLE:  
my_list[2] = 30
```


5. Modifying An Existing Value/Item



6. Inserting a Value/Item

6. Inserting a Value/Item

```
In IDLE:  
my_list.insert(1, 200)
```


6. Inserting a Value/Item

The bucket id

In IDLE:
my_list.insert(1, 200)

A diagram illustrating the insertion of a value into a list. A pink box highlights the number '1' in the code 'my_list.insert(1, 200)'. A pink arrow points from this box to a text box above it that says 'The bucket id'.

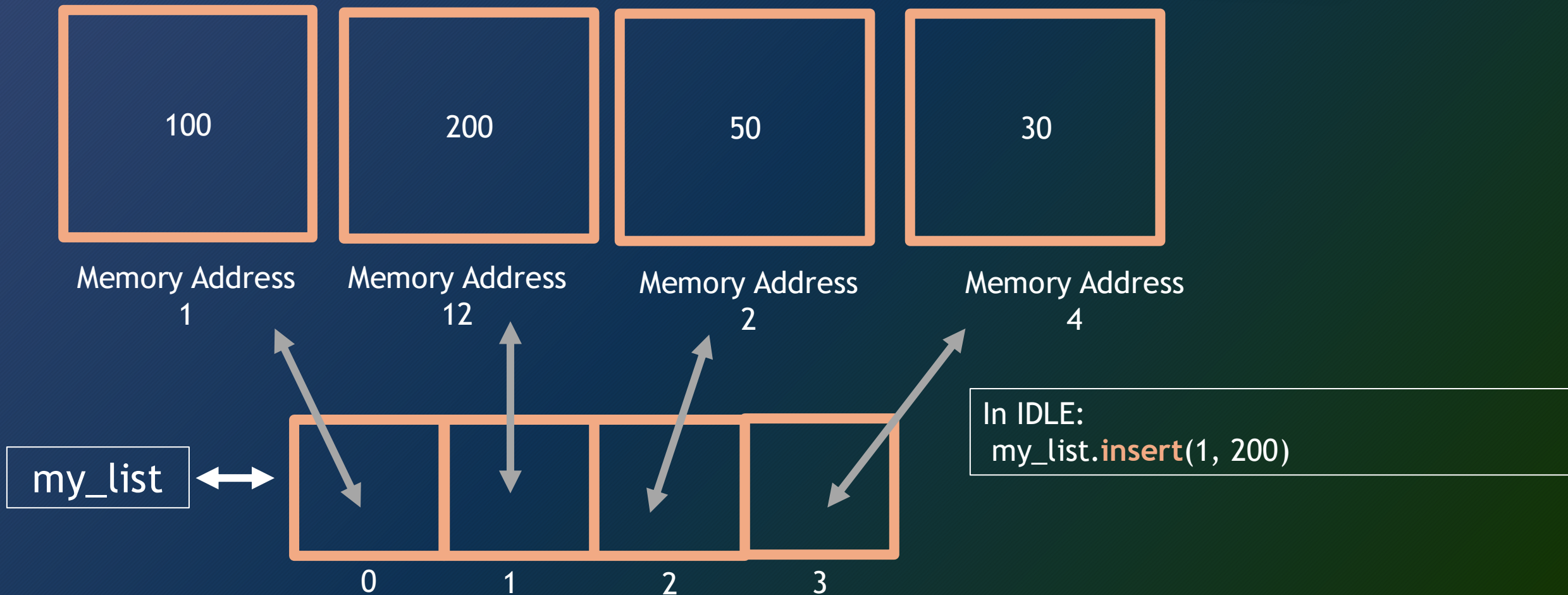
6. Inserting a Value/Item

The value/item to add

In IDLE:
my_list.insert(1, 200)

A diagram illustrating the insertion of a value into a list. A pink box highlights the number 200 in the code snippet 'my_list.insert(1, 200)'. A pink arrow points from this box to a separate pink box above it containing the text 'The value/item to add'.

6. Inserting a Value/Item

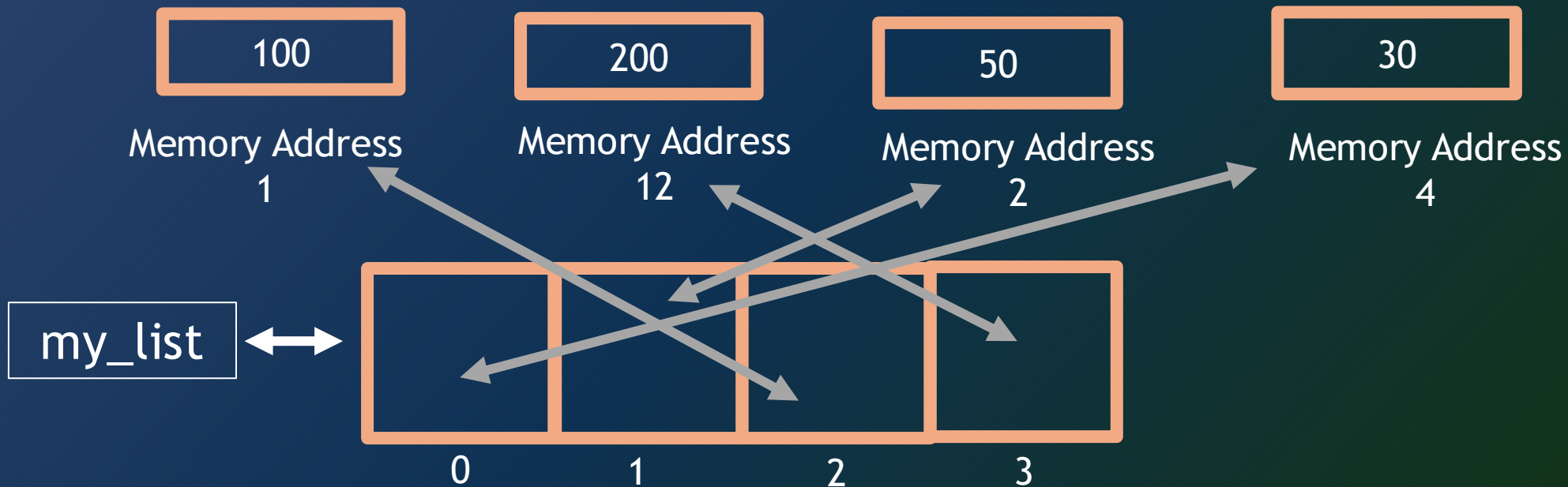


7. Other Methods That Modify In Place

- `Sort()`
 - Current list (`my_list`): `[100, 200, 50, 30]`
 - `my_list.sort()`
 - After Python executes, `my_list`: `[30, 50, 100, 200]`

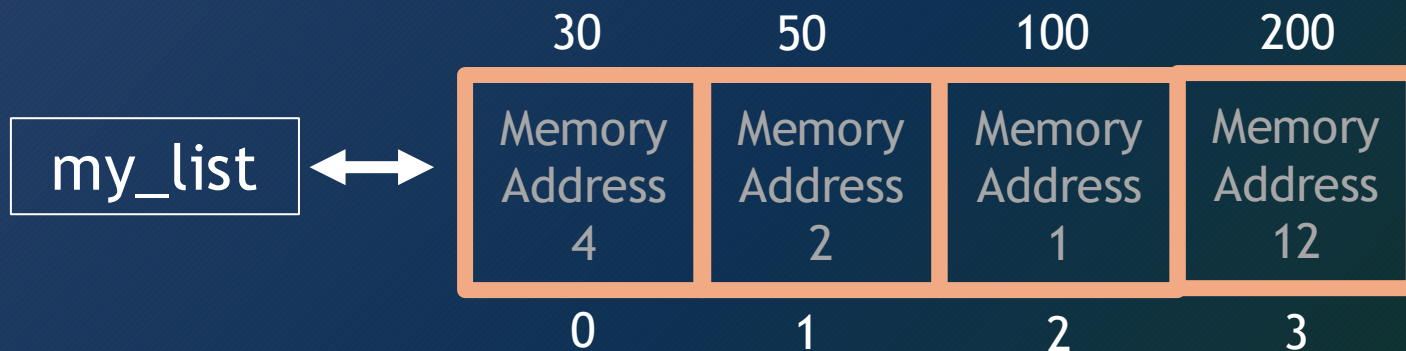
7. Other Methods That Modify In Place

- Sort()
 - Current list (my_list): [100, 200, 50, 30]
 - my_list.sort()
 - After Python executes, my_list: [30, 50, 100, 200]



7. Other Methods That Modify In Place

- `Sort()`
 - Current list (`my_list`): `[100, 200, 50, 30]`
 - `my_list.sort()`
 - After Python executes, `my_list`: `[30, 50, 100, 200]`



Note

- Sort()
- Append()
- Insert(bucket ID, valueToAdd)
- Pop(bucket ID)
- Remove(Value)

None of these methods return a value (except pop).
No copies of `my_list` is made.

Part 2: References and Aliasing

Activity

Activity Continued

```
new_list_1 = new_list_2
```

149

Memory Address 13

50

Memory Address 122

Activity Continued

```
new_list_1 = new_list_2
```

149

Memory Address 13

50

Memory Address 122

220

Memory Address 6

9. Reference and Aliases

In Python, **referencing** is the standard way variables work (labels pointing to objects).

Aliasing is the specific result of having more than one label pointing to the same mutable object.

9. Reference and Aliases

In Python, **referencing** is the standard way variables work (labels pointing to objects).

Aliasing is the specific result of having more than one label pointing to the same mutable object.

In IDLE,

```
list_a = [1, 2, 3] # Referencing  
list_b = list_a   # This is Aliasing!
```


10. Reference and Aliases - A Side-Effect 1

Because `list_a` and `list_b` refer to the same data, any mutation is visible to both.

In IDLE,

```
list_b.append(99)
```

```
print(list_a) # Output: [1, 2, 3, 99]
```

```
print(list_b) # Output: [1, 2, 3, 99]
```


10. Reference and Aliases - To Counter the Side Effect 1

If we actually want a separate list so we can change one without affecting the other, we must **copy** the object rather than just assigning it.

In IDLE,

```
list_a = [1, 2, 3]  
list_b = list_a.copy() # Now they are independent
```

```
list_b.append(99)  
print(list_a) # Output: [1, 2, 3] (Safe!)  
print(list_b) # Output: [1, 2, 3, 99]
```


11. Reference and Aliases - A Side-Effect 2

When we pass a list into a function we aren't passing a "copy" of the list; we are passing a **reference** to the exact same structure/collection in memory.

In IDLE,

```
def mark_as_done(tasks):  
    # 'tasks' is now an alias for 'my_todo_list'  
    tasks.append("Done")  
    print(f"Inside function: {tasks}")
```

```
my_todo_list = ["Email boss", "Buy milk"]  
  
mark_as_done(my_todo_list)  
  
# The change persisted!  
print(f"Outside function: {my_todo_list}")  
# Output: ['Email boss', 'Buy milk', 'Done']
```


11. Reference and Aliases - To Counter the Side Effect 2

Either will work.

Inside the function

```
def mark_as_done_v2(tasks):  
    # 'tasks' is now an alias for  
    # 'my_todo_list'  
    local_tasks = tasks.copy()  
    #local_tasks = tasks[:]  
    local_tasks.append("Done")  
    print(f"Inside function:  
{local_tasks}")  
    #return local_tasks
```

Outside the function

```
my_todo_list = ["Email boss", "Buy milk"]  
  
mark_as_done(my_todo_list.copy())  
#mark_as_done(my_todo_list[:])  
  
# The change persisted!  
print(f"Outside function: {my_todo_list}")  
# Output: ['Email boss', 'Buy milk', 'Done']
```


Exercises

Questions:

7.2.1

7.2.3

7.2.3 and 7.2.4

7.2.2

7.2.5

Part 3: Looping through Lists

12. Index-based Loops

If We want to change the list

```
prices = [10, 20, 30]
```

```
for i in range(len(prices)):
```

```
    prices[i] = prices[i] * 1.1
```

```
# This replaces the pointer at index 'i'
```


13. The Value Loop Trap

The Failure: `item = new_value` only moves the **temporary label** to a new object. It does not change the pointer inside the list.

If We want to change the list

```
nums = [1, 2, 3]
```

```
for n in nums:
```

```
    n = 99 # Only moves the label 'n'; 'nums' remains [1, 2, 3]
```


14. Popping and Removing during iteration

Modifying a list's **length** (using `.pop()` or `.remove()`) while we are currently iterating over it is dangerous because Python's for loop tracks the **index**, not the items.

- **The "Skip" Bug:** If we delete the item at index 0, every other item shifts left to fill the gap. The loop then moves to index 1, but the item that was at index 1 is now at index 0. We just skipped an element because we didn't account for the "internal shuffle."
- **The IndexError:** If we are using a fixed range (like `range(len(my_list))`) and shrinking the list, eventually our loop will try to access an index that no longer exists in the shortened list.

14. Safe Solutions to List Modification Inside a Loop

- **The while loop**

We take manual control over the index. We only increment the index if we don't delete an item. If we do delete an item, we stay at the same index because the "next" item has just shifted into our current spot.

- **Iterate over a copy**

We use `for item in my_list[:]`. By doing this, we are looking at a "photo" (a shallow copy) of the list to decide what to do, while performing the actual deletions on the original physical list. Since the loop is following the copy, the shifting in the original doesn't trip us up.

Part 4: List Comprehensive

Example 1

Regular Code

```
evens = [ ]  
for i in range(15):  
    evens.append(2 * i)
```

With List Comprehensive

```
evens = [2 * i for i in range(15)]
```


Example 1

Regular Code

```
evens = [ ]  
for i in range(15):  
    evens.append(2 * i)
```

With List Comprehensive

```
evens = [2 * i for i in range(15)]
```

```
evens = [ ]  
for i in range(15):  
    evens.append(2 * i)
```

Diagram illustrating the transformation of the regular code into a list comprehension. Red boxes highlight the expression `2 * i` in the `append` call and the final list comprehension `[2 * i for i in range(15)]`. Arrows show the mapping from the loop's iteration to the list comprehension's elements.

```
evens = [2 * i for i in range(15)]
```


Example 2

Regular Code

```
evens = [ ]  
for i in range(15):  
    if 2 * i % 6 != 0:  
        evens.append(2 * i)
```

With List Comprehensive

```
evens = [2 * i for i in range(15) if 2 * i % 6 != 0]
```


Example 2

Regular Code

```
evens = [ ]
for i in range(15):
    if 2 * i % 6 != 0:
        evens.append(2 * i)
```

With List Comprehensive

```
evens = [2 * i for i in range(15) if 2 * i % 6 != 0]
```

```
evens = [ ]
for i in range(15):
    if 2 * i % 6 != 0:
        evens.append(2 * i)
```

The diagram illustrates the transformation of the regular code into a list comprehension. Red boxes highlight the components of the regular code and the resulting list comprehension, with arrows indicating the mapping between them:

- The `evens = [ ]` line is boxed in red.
- The `for i in range(15):` line is boxed in red.
- The `if 2 * i % 6 != 0:` line is boxed in red.
- The `evens.append(2 * i)` line is boxed in red.
- The resulting list comprehension `evens = [2 * i for i in range(15) if 2 * i % 6 != 0]` is shown below, with its components boxed in red: `2 * i`, `for i in range(15)`, and `if 2 * i % 6 != 0`.
- Arrows show the mapping: from the `for` loop to the `for` clause, from the `if` condition to the `if` clause, and from the `append` argument to the expression `2 * i`.

Backbone of List Comprehensive

```
evens = [2 * i for i in range(15) if 2 * i % 6 != 0]
```



The list we
want to create

Backbone of List Comprehensive

```
evens = [2 * i for i in range(15) if 2 * i % 6 != 0]
```

The list we
want to create

The value or
item you want
to store in the
list

Backbone of List Comprehensive

```
evens = [2 * i for i in range(15) if 2 * i % 6 != 0]
```

The list we
want to create

The value or
item you want
to store in the
list

Loop logic;
how many
times should
the loop
repeat

Backbone of List Comprehensive

```
evens = [2 * i for i in range(15) if 2 * i % 6 != 0]
```

The list we
want to create

The value or
item you want
to store in the
list

Loop logic;
how many
times should
the loop
repeat

Logic for selecting
items/values for your list
(optional)

Questions